

Laya System 1 Decision Model

Complete Code Walkthrough & Explanation

convaiinnovations/laya | Apache 2.0 | huggingface.co

This document explains every line of every file in the Laya System 1 Model project. It covers what the code does, why each decision was made, and how the pieces fit together.

1. What Is This Project?

Laya ([convalinnovations/laya](https://github.com/convalinnovations/laya)) is a System 1 decision model -- meaning it works like human intuition: fast, single-pass, and structured. Unlike generative models such as ChatGPT or Claude, which produce text token-by-token (System 2 thinking), Laya reads input text and answers typed questions in a single mathematical forward pass through a transformer neural network.

The result is ~33 ms latency (vs. 1-3 seconds for LLMs), zero hallucinations (it never generates text, so it cannot make things up), and calibrated probability outputs for every answer it gives.

2. Project Files

laya_engine.py	Wrapper around the <code>laya</code> Python package. Defines <code>LayaClassifier</code> and the default question schema.
laya_engine_hf.py	Shows how to use the raw Hugging Face transformers API to tokenize text and run a forward pass.
laya_demo.py	Command-line script. Type any text, get a structured decision instantly.
experiment/laya_experiment.ipynb	Step-by-step Jupyter notebook (9 code cells). Runs on local machine or Google Colab.
requirements.txt	Lists all Python packages needed to run the project.
data/tickets_dataset.json	20 sample customer support tickets used in the notebook batch demo.

3. `laya_engine.py` -- Line by Line

This is the core module. It loads the Laya model from Hugging Face and exposes two things: a `LayaClassifier` class and a `get_default_questions()` function. Every other file imports from here.

```
import laya
```

Imports the 'laya' Python package published by Convai Innovations on PyPI. This package internally handles PyTorch, Hugging Face weights, and the decision logic. One import does all the heavy lifting.

```
class LayaClassifier:
```

Defines a Python class -- a reusable blueprint. Wrapping the model in a class means callers write 'model = LayaClassifier()' instead of remembering the laya API.

```
def __init__(self, model_id='convaiinnovations/laya'):
```

`__init__` is the constructor -- it runs automatically when you create a new instance. `model_id` has a default value so you never have to type the full model name. The `self` keyword refers to the instance being created.

```
    self.agent = laya.load(model_id)
```

`laya.load()` downloads model weights from Hugging Face Hub on the first call (~300 MB), then caches them locally so every subsequent call is instant. `self.agent` stores the loaded model on the instance so all methods can access it.

```
def predict(self, state, questions):
```

Defines the predict method. `state` is a dict with the input text (e.g. {'subject': '...', 'body': '...'}). `questions` is a dict describing what decisions you want (department, urgency, escalation).

```
    return self.agent.predict(state, questions)
```

One-line delegation to the real model. No extra logic -- just a clean passthrough to the laya agent. The model returns a dict with 'answers' and 'latency_ms'.

```
def get_default_questions():
```

A standalone function (not inside the class) that returns a pre-built question schema. This is a convenience function -- instead of writing the schema every time, you call `get_default_questions()` and get three typed questions ready to use.

```
    "department": { "type": "choice", ... }
```

The 'choice' type tells Laya to pick exactly ONE label from the criteria dict. The criteria values are plain-English descriptions that help the model understand what each label means during inference. It uses semantic understanding, not keywords.

```
    "urgency": { "type": "score", "criteria": ["low",...] }
```

The 'score' type tells Laya to rank on an ordered scale. Criteria is a list from lowest to highest. The model returns the appropriate level plus a confidence score.

```
    "needs_human_escalation": { "type": "noul", ... }
```

The 'noul' type is a Yes/No boolean decision. The model returns True or False plus a probability. 'noul' stands for 'no utterance label' -- a binary classification without needing named labels like 'yes' and 'no'.

4. `laya_engine_hf.py` -- Line by Line

This file shows the raw Hugging Face transformers API -- no `laya` package needed. It demonstrates what is happening underneath: tokenization, a forward pass through the transformer, and extracting the CLS token embedding. This is useful for understanding how transformers work at a lower level.

```
import torch
```

PyTorch is the deep learning framework that runs the transformer math. All tensor operations (matrix multiplications, attention heads) happen here.

```
from transformers import AutoTokenizer, AutoModel
```

`AutoTokenizer` and `AutoModel` are Hugging Face's generic loaders. The 'Auto' prefix means they automatically detect the correct class for any model on the Hub, so you do not need to import a model-specific class.

```
MODEL_ID = "convaiinnovations/laya"
```

A module-level constant (uppercase by Python convention). Centralizing the model ID means you change it in one place if you ever want to swap models.

```
def load_model(model_id=MODEL_ID):
```

Defines a function to download and cache the model. `model_id` has a default value so callers can write `load_model()` with no arguments.

```
    tokenizer = AutoTokenizer.from_pretrained(model_id)
```

`from_pretrained()` checks your local Hugging Face cache (`~/.cache/huggingface`) first. If not found, it downloads from the Hub. The tokenizer converts raw text into token IDs that the model understands.

```
    model = AutoModel.from_pretrained(model_id, device_map='auto')
```

Loads the model weights. `device_map='auto'` automatically places the model on GPU if one is available, otherwise falls back to CPU -- no manual device management needed.

```
    return tokenizer, model
```

Returns both objects as a tuple so the caller gets everything they need in one call.

```
def tokenize(tokenizer, text):
```

A thin wrapper around the tokenizer. Separating this into its own function makes the code readable -- each function does exactly one thing.

```
    return tokenizer(text, return_tensors='pt')
```

Calling the tokenizer like a function processes the text. `return_tensors='pt'` means 'return PyTorch tensors'. The result is a dict with `input_ids`, `attention_mask`, etc.

```
def forward_pass(model, inputs):
```

Runs one single pass through all transformer layers. This is the core computation.

```
    with torch.no_grad():
```

A context manager that disables gradient tracking. During training, PyTorch records every operation to compute gradients. For inference we do not need gradients, so disabling them saves memory and runs faster.

```
        outputs = model(**inputs)
```

`**` unpacks the inputs dict as keyword arguments (`input_ids=...`, `attention_mask=...`). This is identical to calling `model(input_ids=..., attention_mask=...)` explicitly.

```
    return outputs.last_hidden_state
```

`last_hidden_state` is a 3-D tensor: `[batch_size, sequence_length, 768]`. 768 is the embedding dimension -- every token gets a 768-number vector representing its meaning in context.

```
def cls_embedding(hidden_state):
```

Extracts the embedding for the [CLS] (Classification) token -- always the first token.

```
return hidden_state[0][0]
```

`hidden_state[0]` selects the first item in the batch. `[0]` again selects token position 0, which is [CLS]. By BERT/transformer convention, [CLS] holds a learned summary representation of the entire input sequence.

5. laya_demo.py -- Line by Line

This is the command-line entry point -- the simplest way to use the model. You run it from the terminal, optionally pass any text as an argument, and get a structured decision printed back.

Usage examples:

```
python laya_demo.py
python laya_demo.py "Double charged on invoice #INV-88912"
python laya_demo.py "Unrecognized login from foreign IP 203.0.113.42"
```

`import sys`

Python standard library. Gives access to `sys.argv` -- the list of words typed in the terminal when running the script.

`from laya_engine import LayaClassifier, get_default_questions`

Imports both things we need from `laya_engine.py`. `LayaClassifier` loads the model, `get_default_questions()` returns the three-question decision schema.

`text = " ".join(sys.argv[1:]) if len(sys.argv) > 1 else (...)`

`sys.argv` is always a list. `sys.argv[0]` is the script name itself. `sys.argv[1:]` is everything the user typed after the script name. `' '.join()` glues multiple words back into one string. If no argument was given, a default example is used.

`model = LayaClassifier()`

Creates an instance of `LayaClassifier`. The constructor runs, which calls `laya.load()` which downloads/loads the model weights.

`questions = get_default_questions()`

WHAT THIS DOES: `get_default_questions()` returns a Python dict with three pre-defined typed questions: (1) 'department' -- a choice question asking which team should handle the ticket, with four labelled options; (2) 'urgency' -- a score question ranking severity from low to critical; (3) 'needs_human_escalation' -- a yes/no boolean. WHY IT EXISTS: Rather than making you write this schema from scratch every time you use `laya_demo.py`, it provides a ready-made set of questions that covers the most common triage decisions for a support ticket system. You could replace this with any custom questions for your own use case.

`state = {"subject": text[:80], "body": text}`

Laya expects the input as a dict called 'state'. The subject is capped at 80 characters (like a ticket title). The body gets the full text. `text[:80]` is Python slice notation -- characters 0 through 79.

`result = model.predict(state, questions)`

Runs one forward pass through the model. Returns a dict with two keys: 'answers' (one entry per question) and 'latency_ms' (time taken in milliseconds).

`dept = answers["department"]`

Extracts the department sub-answer. `dept` is a dict with: 'choice' (the predicted label), 'confidence' (0.0-1.0), 'probabilities' (a score for every possible label).

`bar = "#" * int(prob * 40)`

Creates an ASCII progress bar. `int(0.974 * 40) = 38`, so `'#' * 38` gives a row of 38 hash characters. A visual representation of the probability without needing any charting library.

`print(f"{label:<20} {prob * 100:>5.1f}% {bar}")`

f-string formatting: `<20` left-aligns the label in a 20-character field. `>5.1f` right-aligns the percentage with 1 decimal place in a 5-character field. This keeps all rows neatly aligned in the terminal.

6. experiment/laya_experiment.ipynb -- Step by Step

The notebook has 9 code cells and 10 markdown (text) cells. It is self-contained -- you can upload it directly to Google Colab and run it without installing anything locally. Here is what each code cell does:

Cell 1 -- pip install

```
!pip install transformers torch laya pandas matplotlib --quiet
```

The ! prefix runs a shell command inside the notebook. This installs all required packages. --quiet suppresses the verbose output. Run this only once.

Cell 2 -- Load model

```
AutoTokenizer.from_pretrained(model_id) + AutoModel.from_pretrained(model_id, device_map='auto')
```

Downloads model weights from Hugging Face Hub on first run (~300 MB). Subsequent runs use the local cache and are instant.

Cell 3 -- Tokenize

```
tokenizer(sample_text, return_tensors='pt')
```

Converts the raw string into a PyTorch tensor of integer token IDs. Also shows the human-readable token list so you can see how the model reads text.

Cell 4 -- Forward pass

```
with torch.no_grad(): outputs = model(**inputs)
```

Runs the text through all 12 transformer layers in one shot. Prints the output shape [1, num_tokens, 768] -- 768 values per token.

Cell 5 -- Load via laya package

```
agent = laya.load('convaiinnovations/laya')
```

Loads the same model via the high-level laya API. Reuses cached weights -- no second download needed.

Cell 6 -- Define state and questions

```
state = {'subject': ..., 'body': ...} questions = {'department': ..., ...}
```

Sets up the input text and the typed question schema. The questions dict tells Laya exactly what decisions to make.

Cell 7 -- Run inference

```
result = agent.predict(state, questions)
```

One forward pass. Returns department, urgency, escalation flag, and latency in ms.

Cell 8 -- Visualize probabilities

```
matplotlib bar chart showing calibrated probability for all four departments.
```

```
matplotlib.use('Agg')
```

sets a headless backend that works on any server or Colab session.

Cell 9 -- Batch testing

```
for ticket in tickets: agent.predict(state_t, questions)
```

Processes 5 different tickets and collects results into a pandas DataFrame. Shows how to scale from a single prediction to processing many inputs.

7. Why System 1? The Architecture Explained

Understanding why Laya is fast and reliable requires understanding the difference between the two types of AI reasoning:

System 2 Models (ChatGPT, Claude, Llama)

These are autoregressive generative models. When you ask them to classify a ticket, they generate a response one token at a time. To produce the word 'billing' as a classification answer, the model runs its full forward pass once per token: first for 'b', then 'i', then 'l', and so on. Each forward pass takes ~5-10 ms, and a full response can be 50-200 tokens, totalling 1,000-3,000 ms.

This also means the output is unstructured text. If you need a JSON response, the model might hallucinate extra fields, forget quotes, or produce invalid JSON.

System 1 Model (Laya)

Laya is non-autoregressive. It reads the entire input text in one forward pass through a 12-layer transformer (identical architecture to BERT). Instead of generating text, it uses the learned [CLS] token embedding to directly compute probability distributions over your predefined labels. There is no generation loop. One pass = one answer = ~33 ms.

Because it never generates text, it cannot hallucinate. The output is always valid structured JSON matching exactly the schema you defined in your questions dict.

Calibrated Probabilities (RLCD)

Most classifiers give you overconfident probabilities. Laya is trained with Reinforcement Learning from Calibrated Demonstrations (RLCD), a technique that penalizes overconfidence and rewards honest uncertainty. This means when Laya says 96.2% billing, it is genuinely that confident -- not just outputting a high softmax score because it is the top prediction.